

Implemented openSSH instead of plink for cross-platform compatibility purposes
Jesse Abdul authored 24 minutes ago

Staff Info App - Technical Documentation.md 11.69 KiB

Staff Information App Technical Documentation

Overview

The Staff Information Application (SIA) is a web app that provides access to the PIFSC Oracle personnel database for information that does not contain sensitive information or PII. There are two web pages; one provides a sortable tabular staff directory listing and the other shows a formatted organization hierarchy chart. The web app is implemented as a docker container.

Resources

- Version Control Information:
  - URL: https://picgitlab.nmfs.local/omi-apps/staff-directory.git
  - Application: 1.2 (Git tag: staff_info_app_v1.2)
- End-User Documentation

Application Access

- *Note: by default the staff directory web page is loaded when the application URLs are loaded
- Development: https://picapps-dev.nmfs.local/staff-directory/
- Test: https://picapps-test.nmfs.local/staff-directory/
- Production: https://picapps.nmfs.local/staff-directory/

Requirements

- Docker Host
  - Apache 2.x, PHP 8.x
  - Docker host and reverse proxy configured to allow web requests to the docker host via port 80 to the web application on port 5006
  - Docker secrets
- Client Machine (for deployments)
  - ssh is setup to work with CAC authentication
  - ssh is configured to specify the username in the ~/.ssh/config file for each docker host (e.g. pifsc-dev-docker-01-as for the dev docker host)
- User:
  - Any modern web browser
  - PIFSC network connection

Database Instances

- For the development docker and database instances the abbreviation used is "dev"
- For the test/QA docker and database instances the abbreviation used is "test"
- For the production docker and database instances the abbreviation used is "prod"

Dependencies

- The SIA utilizes the PHP Shared Library for Oracle database interactions
  - This application was developed with version 1.16 (tag: php_shared_libary_v1.16) of the PHP Shared Library

Docker Image Details

The Docker image is built from the official Oracle InstantClient on OracleLinux 8 Docker image. Apache and PHP are installed on the base image.

Implementation

- There are two PHP pages that are accessed directly via web browser, one for the staff directory (staff-directory.php) and one for the organization chart (org-chart.php)
- **Shared Elements**

    - There is a navigation menu at the top of both web pages that provides links to the two pages, the link for the current page is grayed out and the corresponding link is not enabled
- **Staff Directory**
    - The staff directory displays all current PIFSC personnel in a table with several columns of information. Each column heading is implemented as a link that provides sorting capabilities with the appropriate icons showing the current sort order for the column (if any) or if there is no sorting applied to the given column it is shown with a general sort icon.
    - Email and phone numbers are implemented as links.
- **Organization Chart**
    - This web page displays all of the current PIFSC personnel in a hierarchical diagram using HTML and css
    - The Directors Office is shown at the top of the page
        - The Divisions are shown below along with all Subdivisions, Programs and Subprograms nested below the corresponding Divisions.
    - Each individual is shown below their corresponding organizational unit with their abbreviated position with indicators based on their employer (e.g. (c) for contractors, (CI) for cooperative institute employees)
    - Features:
        - Tooltips are implemented for all personnel. When a given person's email or phone numbers are stored in the database, links for each are available in the tooltip.

Repository Folder Structure

- The httpd folder contains the Apache configuration
- The php folder contains the PHP configuration
- The deployment_scripts folder contains scripts to prepare and deploy the docker project
- The docs folder contains documentation for the project
- The src folder contains all source code for the project
- The backend folder contains backend PHP include files for the project
- The frontend folder contains the PHP web pages that are directly requested via web browser
    - The img folder contains the client-side images used in the web pages
    - The js folder contains the client-side JavaScript code used in the web pages
    - The style folder contains the client-side css used in the web pages
    - The webfonts folder contains the client-side web fonts used by the css in the web pages
- **Repository Folder Diagram:**

```
.
|--- docker
|    |--- docker-compose.yml
|    |--- Dockerfile
|    |--- config
|    |    |--- httpd
|    |    |--- php
|    |--- deployment_scripts
|    |--- src
|    |    |--- backend
|    |    |    |--- dependencies
|    |    |    |--- *.php
|    |    |--- frontend
|    |    |    |--- *.php
|    |    |    |--- img
|    |    |    |--- js
|    |    |    |--- style
|    |    |    |--- webfonts
|--- docs
```

Setup

- Clone the SIA repository
- Within the project repository create the necessary bash files with the database credentials in each database instance (e.g. secrets.sh in the dev folder for the development database instance)
    - *Note: There is a secrets template file that can be used to create the secrets.sh file for each database instance
- Within the project repository create the necessary configuration bash files with the database connection information for each database instance
    - *Note: There is a database instance configuration template file that can be used to create the deploy_config.[ENV].sh file (e.g. deploy_config.dev.sh for the development database instance)
- **Development Machine (Docker Desktop)**

- (One-time setup) Setup docker swarm: `docker swarm init`

Docker Deployment

- The following instructions are intended for different deployment environments and use cases
- *Note: the steps shown here were developed and tested on a Windows computer and the bash scripts were executed using git-bash
- **Local Docker Desktop**
  - **Local Development**
    - *Note: this use case is intended for local docker desktop development that uses Docker Compose and maps local folders within the working copy of the repository to facilitate rapid application development
    - Execute the deploy_container_to_local.sh script with the following arguments:
      - Argument 1: 1
      - Argument 2: the abbreviation for the given database instance listed in the Database Instances section (e.g. dev for development)
    - The script will perform the following high-level processes:
      - load the corresponding database credentials as environment variables within the docker container
      - Clone the container dependencies and copy them into the designated folder (dependencies) in the working directory
      - Build and run the docker container
  - **Local Testing**
    - *Note: this use case is intended for local docker desktop development/testing that uses docker swarm and docker secrets to help ensure that the application can be deployed to a remote docker host
    - Execute the deploy_container_to_local.sh script with the following arguments:
      - Argument 1: 0
      - Argument 2: the abbreviation for the given database instance listed in the Database Instances section (e.g. test for test/QA)
    - The script will perform the following high-level processes:
      - Specify the docker secrets for the docker application
      - Clone the container dependencies and copy them into the designated folder (dependencies) in the working directory
      - Build and run the docker container within the docker desktop swarm
- **Remote Docker Host**
  - *Note: this use case is intended for deployment to a remote docker host that uses docker swarm and docker secrets to help ensure that the database credentials are secured against unauthorized access
  - Execute the deploy_container_to_host.sh script with the following arguments:
    - Argument 1: the abbreviation for the given database instance listed in the Database Instances section (e.g. prod for production)
  - The script will perform the following high-level processes:
    - Clone the SIA repository within the docker host to a working directory
    - Clone the container dependencies and copy them into the designated folder (dependencies) in the working directory
    - Switch to the designated docker user that has permissions to run docker commands:
      - Copy the working repository into the designated docker folder
      - Specify the docker secrets for the docker application
      - Build and run the docker application
    - Delete the working copy of the repository

Docker Folder Structure

- The docker folder structure closely mirrors the Repository Folder Structure except for a few notable differences:
  - The /var/www/html folder is populated with the contents of the frontend folder from the repository
  - The /usr/src/app folder is populated with the contents of the backend folder from the repository
  - The /etc/php.d folder is populated with the contents of the php folder from the repository
  - The /etc/httpd/conf.d folder is populated with the contents of the php folder from the repository
- **Docker Folder Diagram**

```
.
|--- /var/www/html
|    |--- img
|    |--- js
|    |--- style
|    |--- webfonts
|    |--- *.php
|--- /usr/src/app
|    |--- *.php
|    |--- dependencies
|    |    |--- *.php
|--- /etc/php.d
```

```
|--- /etc/httpd/conf.d
```

Security Features

- Global Security Features:
  - SQL queries use prepared statements and bind variables to prevent SQL Injection Attacks
  - Database values are sanitized using htmlspecialchars() before being included in the HTML web page content to prevent Cross Site Scripting (XSS) attacks
  - Database credentials are managed within the docker swarm using docker secrets to help ensure the credentials are secured against unauthorized access
- Organization Chart:
  - Tooltip content is retrieved from the database and stored in a javascript array of objects, when a given tooltip is activated the content in the corresponding object variable is sanitized before it is displayed
- Staff Directory:
  - User Input for links (used in sorting functionality) is validated against a list of acceptable values and if there